

Diagnosi del codi per a una botiga ràpida i eficient

Dades clau

Mètrica	Valor
Fitxers JS/JXS	296
Línies totals codi	120.300
JS bundle total (producció)	3.6 MB
CSS bundle	92 KB
Assets públics	677 MB (2.738 imatges, 411 MB en webp)
PDPs duplicats	64 fitxers x 736 línies = 47.000 línies

Problemes prioritzats (d'alta a baixa impacte)

1. App.jsx és un monòlit de 6.082 línies — Impacte: ALT

- Conté 224 hooks (useState/useEffect/useMemo/useCallback)
- Importa ~130 lazy components però tot el codi de routing, layout, contexts, i lògica de negoci viu en un sol fitxer
- Qualsevol canvi força re-renders globals
- **Solució:** Extreure lògica a hooks separats (useAppRoutes, useAppLayout, useAppDebug) i reduir App.jsx a < 500 línies

2. 64 PDPs duplicats — 47.000 línies de codi idèntic — Impacte: ALT

- Cada PDP (src/pages/products/*.jsx) és una còpia de 736 línies amb només 4-5 línies canviades (slug, nom, col·lecció)
- Ocupen ~47 KB de codi font però el patró és insostenible per manteniment
- **Solució:** Un sol component ProductDetailPageTemplate que rep collectionSlug + productRoute via props o URL params. Reduir 47.000 línies a ~800

3. FullWideSlideHeader.jsx — 3.900 línies, 123 hooks — Impacte: ALT

- És el component més complex. Gestiona pàgina 1, pàgina 2, cistell, perfil, calibratge, selectors, overlays
- 273 KB de bundle (66 KB gzip)
- **Solució:** Extreure pàgines 3 (cistell) i 4 (perfil) a components separats. Ja hem començat amb pàgina 2 (MegaslidePagina2). Continuar amb MegaslidePagina3 i MegaslidePagina4

4. Supabase carregat globalment — 124 KB (34 KB gzip) — Impacte: MITJÀ-ALT

- @supabase/supabase-js s'inclou al chunk supabase però es carrega sempre, fins i tot a la home on no es necessita
- 23 fitxers l'importen, molts són pàgines admin
- **Solució:** Lazy-load dinàmic de l'API de supabase dins dels components que la necessiten (admin, checkout, fulfillment). La home no hauria de carregar supabase

5. Framer Motion — 120 KB (40 KB gzip) usat a totes les rutes — Impacte: MITJÀ

- 36 fitxers l'importen
 - `App.jsx` l'usa per animacions de transició de pàgines (31 usos de `motion./ AnimatePresence`)
 - **Solució:** Per la botiga pública, les transicions es poden fer amb CSS (opacity + transform). Reservar framer-motion per a pàgines admin/ demo on és realment útil
6. `index.js` — 384 KB (94 KB gzip) — chunk principal massa gran — Impacte: MITJÀ
- És el chunk que es carrega sempre. Conté codi compartit entre totes les rutes
 - Probablement inclou codi de components compartits que no totes les pàgines necessiten
 - **Solució:** Revisar `manualChunks` al `vite.config.js` i separar codi compartit en chunks més granulars
7. 677 MB d'assets públics sense CDN ni optimització — Impacte: MITJÀ
- 2.738 imatges, majoritàriament webp (411 MB) — format correcte
 - Però es serveixen des del mateix servidor. Sense CDN, sense cache headers configurats
 - **Solució:**
 - Servir assets des d'un CDN (Cloudflare, Netlify CDN)
 - Configurar `Cache-Control: public, max-age=31536000, immutable` per a assets amb hash
 - Considerar un servei d'imatges (Cloudinary, Imgix) per servir mides sota demanda
8. Sense lazy loading d'imatges als components clau — Impacte: MITJÀ
- `FullWideSlideHeader` fa preload de 7 imatges però no usa `loading="lazy"` en els `<img>` de la franja
 - `MegaStripePanel` renderitza 14 tiles sense lazy loading
 - **Solució:** Afegir `loading="lazy"` i `fetchpriority="high"` només a la primera imatge visible
9. `MainHeader.jsx` — 1.143 línies, carregat sempre — Impacte: MITJÀ
- Es carrega a totes les pàgines (inclòs a `AppProd.jsx`)
 - Probablement conté lògica del mega-menu que només es necessita a la home
 - **Solució:** Extreure el mega-menu a un component lazy-load separat
10. `TdpVariantsGallery` — 296 KB (84 KB gzip) — el chunk més gran — Impacte: BAIX-MITJÀ
- És un component de construcció/admin, però es carrega via lazy import
 - Tot i així, 296 KB és excessiu per un component
 - **Solució:** Revisar si es pot dividir en sub-chunks o si conté codi que es podria externalitzar
11. `App.jsx` vs `AppProd.jsx` — dues apps separades — Impacte: BAIX
- `main.jsx` carrega `App.jsx` (dev), `main-prod.jsx` carrega `AppProd.jsx` (prod)
 - `AppProd.jsx` és net (164 línies, lazy loading correcte)
 - Però `App.jsx` té 6.082 línies i inclou tot el codi dev/admin

- **Solució:** Si la botiga pública usa `AppProd.jsx`, el codi `dev d'App.jsx` no afecta la producció. Verificar quin entry point s'usa en deploy

12. Contexts carregats globalment — possible overhead — Impacte: BAIX

- `ProductProvider`, `CartProvider`, `WishlistProvider`, `AdminProvider`, `AdminToolsProvider`, `GridDebugProvider`, `ToastProvider`
- Tots es carreguen al `main.jsx` abans de qualsevol ruta
- `AdminProvider` i `AdminToolsProvider` no són necessaris per a la botiga pública
- **Solució:** Moure contexts admin dins del layout admin (`AdminStudioLayout`)

13. CSS global — 92 KB — Impacte: BAIX

- Un sol fitxer CSS de 92 KB. Probablement Tailwind + estils globals
- No és crític però es podria purgar CSS no usat

Resum d'accions per prioritat

#	Acció	Esforç	Impacte
1	Unificar 64 PDPs en un sol component template	Mitjà	Alt
2	Extreure pàgines 3 i 4 de <code>FullWideSlideHeader</code>	Mitjà	Alt
3	Reduir <code>App.jsx</code> (extreure hooks)	Mitjà	Alt
4	Lazy-load Supabase fora de la botiga pública	Baix	Alt
5	Substituir Framer Motion per CSS a la botiga pública	Baix	Mitjà
6	Configurar CDN + cache per assets	Baix	Mitjà
7	Lazy loading d'imatges als components stripe	Baix	Mitjà
8	Moure contexts admin dins del layout admin	Baix	Mitjà
9	Extreure mega-menu de <code>MainHeader</code>	Baix	Mitjà
10	Purgar CSS no usat	Baix	Baix